GPU-resident CUDA BHO benchmark (Google Colab)

This notebook runs a **real CUDA implementation** on an NVIDIA GPU and compares it with sequential CPU, OpenMP CPU, and CUDA implementations using the same search equations.

What executes on the GPU: population initialization, candidate generation, objective-function evaluation, and greedy selection. The population and fitness arrays stay in device memory throughout the optimization loop; only the final best score is copied back. The global-best scan is also launched on the GPU (currently a simple one-thread scan, which can be optimized later).

Important algorithm note: the original BHO source includes K-means representative/group initialization, QPRS reassignment, and Explorpolis diversity admission. This notebook implements a GPU-resident **BHO-inspired variant of its best-guided/peer-guided search**, not a line-by-line CUDA port of every mechanism in the paper. Its CPU and CUDA versions are matched to each other for fair timing and score comparisons. Do not describe it as an exact reproduction of the paper's full BHO algorithm.

1. Select an NVIDIA GPU runtime

In Colab: **Runtime → Change runtime type → T4 GPU** (or another available GPU), then run the cells from top to bottom. This notebook deliberately stops if CUDA tools or an NVIDIA GPU are unavailable.

```
import torch
import time

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

    # Allocate data directly on the GPU
    a = torch.randn((4096, 4096), device="cuda")
    b = torch.randn((4096, 4096), device="cuda")

    torch.cuda.synchronize()
    start = time.perf_counter()

    for _ in range(30):
        c = torch.matmul(a, b)

    torch.cuda.synchronize()
    elapsed = time.perf_counter() - start

    print(f"GPU computation completed in {elapsed:.3f} seconds")
    print(
        "GPU memory allocated:",
        round(torch.cuda.memory_allocated() / 1024**2, 1),
        "MiB"
    )
else:
    print("CUDA is unavailable to PyTorch.")
```

```
PyTorch: 2.11.0+cu130
CUDA available: True
GPU: Tesla T4
GPU computation completed in 1.115 seconds
GPU memory allocated: 200.1 MiB
```

```
import os, shutil, subprocess, time
import pandas as pd
import matplotlib.pyplot as plt
from io import StringIO

print("nvidia-smi:", shutil.which("nvidia-smi"))
print("nvcc:", shutil.which("nvcc"))
if shutil.which("nvidia-smi") is None:
    raise RuntimeError("No NVIDIA GPU detected. Choose Runtime > Ch
if shutil.which("nvcc") is None:
    raise RuntimeError("nvcc is unavailable in this runtime. Restar
subprocess.run(["nvidia-smi"], check=True)
```

```
nvidia-smi: /opt/bin/nvidia-smi
nvcc: /usr/local/cuda/bin/nvcc
CompletedProcess(args=['nvidia-smi'], returncode=0)
```

```
!nvidia-smi -l 1
```

```
|                                         |
+-----------------------------------------+------------------------

+-----------------------------------------------------------------------
| Processes:
|  GPU   GI   CI              PID   Type   Process name
|        ID   ID
|=======================================================================
|    0   N/A  N/A            1277      C   /usr/bin/python3
+-----------------------------------------------------------------------
Fri Oct 23 02:44:38 2026
+-----------------------------------------------------------------------
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07
+-----------------------------------------+------------------------
| GPU  Name                 Persistence-M | Bus-Id          Disp.A
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage
|                                         |
|=========================================+========================
|   0  Tesla T4                       Off |   00000000:00:04.0 Off
| N/A   40C    P0             25W /   70W |     393MiB /  15360MiB
|                                         |
+-----------------------------------------+------------------------

+-----------------------------------------------------------------------
| Processes:
|  GPU   GI   CI              PID   Type   Process name
|        ID   ID
|=======================================================================
|    0   N/A  N/A            1277      C   /usr/bin/python3
+-----------------------------------------------------------------------
```

```
Fri Oct 23 02:44:39 2026
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07                 |
+-----------------------------------------+------------------------+----------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util |
|                                         |                        |          |
|=========================================+========================+==========|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |        0 |
| N/A   40C    P0             25W /   70W |     393MiB /  15360MiB |      0%  |
|                                         |                        |          |
+-----------------------------------------+------------------------+----------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI              PID   Type   Process name                  GPU Memory |
|        ID   ID                                                               Usage |
|=============================================================================|
|    0   N/A  N/A            1277      C   /usr/bin/python3                      |
+-----------------------------------------------------------------------------+
Fri Oct 23 02:44:40 2026
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07                 |
+-----------------------------------------+------------------------+----------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util |
|                                         |                        |          |
```

2. Write the CPU and CUDA source files

The sources are embedded in this notebook, so you do not need to upload separate `.cpp` or `.cu` files.

```
cpu_source = '// CPU baseline for the same GPU-resident BHO-inspire

cuda_source = '// GPU-resident BHO-style optimizer (CUDA C++)\n// P

with open("bho_cpu_comparison.cpp", "w", encoding="utf-8") as f:
    f.write(cpu_source)
with open("bho_cuda_gpu_resident.cu", "w", encoding="utf-8") as f:
    f.write(cuda_source)

print("CPU source bytes:", os.path.getsize("bho_cpu_comparison.cpp"
print("CUDA source bytes:", os.path.getsize("bho_cuda_gpu_resident.
```

```
CPU source bytes: 4556
CUDA source bytes: 8377
```

```
# Create an OpenMP implementation from the same C++ algorithm and b
# Only independent per-hunter loops are parallelized; best-index sc
omp_source = cpu_source.replace(
    """        for(int i=0;i<N;i++){
            double* xi=&X[(size_t)i*D];""",
    """           #pragma omp parallel for schedule(static)
        for(int i=0;i<N;i++){
            double* xi=&X[(size_t)i*D];"""
).replace(
    """            for(int i=0;i<N;i++){
                const double* xi=&X[(size_t)i*D];""",
    """            #pragma omp parallel for schedule(static)
            for(int i=0;i<N;i++){
                const double* xi=&X[(size_t)i*D];"""
).replace(
    """            for(int i=0;i<N;i++){
                const double* oldx=&X[(size_t)i*D];""",
    """            #pragma omp parallel for schedule(static)
            for(int i=0;i<N;i++){
                const double* oldx=&X[(size_t)i*D];"""
)
if omp_source == cpu_source or omp_source.count("#pragma omp parall
    raise RuntimeError("OpenMP source transformation failed.")
with open("bho_openmp.cpp", "w", encoding="utf-8") as f:
    f.write(omp_source)
print("Created bho_openmp.cpp with", omp_source.count("#pragma omp
```

```
Created bho_openmp.cpp with 3 parallel loops.
```

3. Compile sequential, OpenMP, and CUDA programs

The OpenMP version uses `g++ -fopenmp`. It parallelizes independent population-level loops while keeping the algorithm and benchmark functions aligned with the sequential baseline. CUDA is compiled with `nvcc`.

```
cpu_cmd = ["g++", "-O3", "-std=c++17", "bho_cpu_comparison.cpp", "-
cpu_build = subprocess.run(cpu_cmd, capture_output=True, text=True)
print("$", " ".join(cpu_cmd))
print(cpu_build.stdout)
print(cpu_build.stderr)
cpu_build.check_returncode()

omp_cmd = ["g++", "-O3", "-std=c++17", "-fopenmp", "bho_openmp.cpp"
omp_build = subprocess.run(omp_cmd, capture_output=True, text=True)
print("$", " ".join(omp_cmd))
print(omp_build.stdout)
print(omp_build.stderr)
omp_build.check_returncode()

cuda_cmd = ["nvcc", "-O3", "-std=c++17", "bho_cuda_gpu_resident.cu"
cuda_build = subprocess.run(cuda_cmd, capture_output=True, text=Tru
print("$", " ".join(cuda_cmd))
print(cuda_build.stdout)
print(cuda_build.stderr)
cuda_build.check_returncode()

print("Sequential, OpenMP, and CUDA programs compiled successfully.
```

```
$ g++ -O3 -std=c++17 bho_cpu_comparison.cpp -o bho_cpu


$ g++ -O3 -std=c++17 -fopenmp bho_openmp.cpp -o bho_openmp


$ nvcc -O3 -std=c++17 bho_cuda_gpu_resident.cu -o bho_cuda


Sequential, OpenMP, and CUDA programs compiled successfully.
```

4. Run benchmark trials

All three implementations use the same benchmark functions, dimensions, population size, iteration count, and repeat count. OpenMP thread count can be adjusted with `OMP_NUM_THREADS`.

```
# Recommended first run; increase after checking that everything wo
D = 500
POPULATION = 2048
ITERATIONS = 1000
REPEATS = 3
OMP_THREADS = 4  # Change to the number of CPU threads you want to
os.environ["OMP_NUM_THREADS"] = str(OMP_THREADS)

def run_program(executable, label):
    cmd = ["./" + executable, str(D), str(POPULATION), str(ITERATIO
    print(f"Running {label}: {' '.join(cmd)}")
    proc_env = os.environ.copy()
    proc_env["OMP_NUM_THREADS"] = str(OMP_THREADS)
    proc = subprocess.run(cmd, capture_output=True, text=True, env=
    if proc.stderr:
        print(proc.stderr)
    if proc.returncode != 0:
        print(proc.stdout)
        proc.check_returncode()
    df = pd.read_csv(StringIO(proc.stdout))
    df["implementation"] = label
    return df

cpu_results = run_program("bho_cpu", "CPU")
omp_results = run_program("bho_openmp", "OpenMP")
gpu_results = run_program("bho_cuda", "CUDA GPU")
results = pd.concat([cpu_results, omp_results, gpu_results], ignore
display(results)
print("OpenMP threads:", OMP_THREADS)
```

```
Running CPU: ./bho_cpu 500 2048 1000 3
Running OpenMP: ./bho_openmp 500 2048 1000 3
Running CUDA GPU: ./bho_cuda 500 2048 1000 3
GPU,Tesla T4
```

	function	dimension	population	repeat	best_score	seconds
0	Sphere	500	2048	0	4.998100e+04	8.021843

1	Sphere	500	2048	1	4.259996e+04	7.965113
2	Sphere	500	2048	2	4.460041e+04	7.969410
3	Rastrigin	500	2048	0	4.609861e+03	25.265159
4	Rastrigin	500	2048	1	4.429569e+03	26.236288
5	Rastrigin	500	2048	2	4.660741e+03	25.490386
6	Rosenbrock	500	2048	0	1.938533e+07	8.073418
7	Rosenbrock	500	2048	1	2.941449e+07	8.165974
8	Rosenbrock	500	2048	2	2.406289e+07	8.091106
9	Ackley	500	2048	0	1.581844e+01	25.379633
10	Ackley	500	2048	1	1.633500e+01	25.830189
11	Ackley	500	2048	2	1.585794e+01	25.539839
12	Griewank	500	2048	0	4.342488e+02	32.785023
13	Griewank	500	2048	1	4.129858e+02	32.810698
14	Griewank	500	2048	2	4.249692e+02	32.738786
15	Schwefel	500	2048	0	1.629499e+05	27.443708
16	Schwefel	500	2048	1	1.665792e+05	28.102051
17	Schwefel	500	2048	2	1.590565e+05	27.530909
18	Sphere	500	2048	0	4.998100e+04	6.769564
19	Sphere	500	2048	1	4.259996e+04	6.688946
20	Sphere	500	2048	2	4.460041e+04	6.593341
21	Rastrigin	500	2048	0	4.609861e+03	20.449989
22	Rastrigin	500	2048	1	4.429569e+03	20.699251
23	Rastrigin	500	2048	2	4.660741e+03	20.713304
24	Rosenbrock	500	2048	0	1.938533e+07	6.987413
25	Rosenbrock	500	2048	1	2.941449e+07	6.961298
26	Rosenbrock	500	2048	2	2.406289e+07	6.944481
27	Ackley	500	2048	0	1.581844e+01	20.765834
28	Ackley	500	2048	1	1.633500e+01	20.962821
29	Ackley	500	2048	2	1.585794e+01	20.809544

30	Griewank	500	2048	0	4.342488e+02	23.863939
31	Griewank	500	2048	1	4.129858e+02	23.881511
32	Griewank	500	2048	2	4.249692e+02	23.856296
33	Schwefel	500	2048	0	1.629499e+05	21.615499
34	Schwefel	500	2048	1	1.665792e+05	21.682387
35	Schwefel	500	2048	2	1.590565e+05	21.600127
36	Sphere	500	2048	0	4.998100e+04	1.175769
37	Sphere	500	2048	1	4.259996e+04	1.115905
38	Sphere	500	2048	2	4.460041e+04	1.117005
39	Rastrigin	500	2048	0	4.609861e+03	1.546413
40	Rastrigin	500	2048	1	4.429569e+03	1.548503
41	Rastrigin	500	2048	2	4.660741e+03	1.544282
42	Rosenbrock	500	2048	0	1.938533e+07	1.198156
43	Rosenbrock	500	2048	1	2.941449e+07	1.198912
44	Rosenbrock	500	2048	2	2.406289e+07	1.198830
45	Ackley	500	2048	0	1.581844e+01	1.528457
46	Ackley	500	2048	1	1.633500e+01	1.520372
47	Ackley	500	2048	2	1.585794e+01	1.519709
48	Griewank	500	2048	0	4.342488e+02	2.095433
49	Griewank	500	2048	1	4.129858e+02	2.095007
50	Griewank	500	2048	2	4.249692e+02	2.095897
51	Schwefel	500	2048	0	1.629499e+05	1.693719
52	Schwefel	500	2048	1	1.665792e+05	1.693477
53	Schwefel	500	2048	2	1.590565e+05	1.696281

```
OpenMP threads: 4
```

Next steps: Generate code with `results` New interactive sheet

5. Summary table and speedup

Speedup is `CPU time / CUDA time`. A value above 1 means CUDA is faster. Timing includes the optimization kernels and synchronization; GPU memory allocation is outside the timed loop. Small tests may be dominated by kernel-launch overhead.

```
summary = (results.groupby(["function", "implementation"])
           .agg(mean_best_score=("best_score", "mean"),
                std_best_score=("best_score", "std"),
                mean_seconds=("seconds", "mean"),
                std_seconds=("seconds", "std"))
           .reset_index())
display(summary)

times = summary.pivot(index="function", columns="implementation", v
scores = summary.pivot(index="function", columns="implementation",
comparison = pd.DataFrame(index=times.index)
comparison["CPU mean seconds"] = times["CPU"]
comparison["OpenMP mean seconds"] = times["OpenMP"]
comparison["CUDA mean seconds"] = times["CUDA GPU"]
comparison["Speedup CPU/OpenMP"] = comparison["CPU mean seconds"] /
comparison["Speedup CPU/CUDA"] = comparison["CPU mean seconds"] / t
comparison["CPU mean score"] = scores["CPU"]
comparison["OpenMP mean score"] = scores["OpenMP"]
comparison["CUDA mean score"] = scores["CUDA GPU"]
display(comparison.reset_index())

results.to_csv("bho_cpu_openmp_cuda_raw_trials.csv", index=False)
summary.to_csv("bho_cpu_openmp_cuda_summary.csv", index=False)
comparison.to_csv("bho_cpu_openmp_cuda_comparison.csv")
print("Saved raw trials, summary, and comparison CSV files.")
```

	function	implementation	mean_best_score	std_best_score	mean_
0	Ackley	CPU	1.600379e+01	2.875135e-01	2
1	Ackley	CUDA GPU	1.600379e+01	2.875135e-01	
2	Ackley	OpenMP	1.600379e+01	2.875135e-01	2
3	Griewank	CPU	4.240679e+02	1.066014e+01	3
4	Griewank	CUDA GPU	4.240679e+02	1.066014e+01	
5	Griewank	OpenMP	4.240679e+02	1.066014e+01	2

5	Griewank	OpenMP	4.240679e+02	1.066014e+01	2
6	Rastrigin	CPU	4.566724e+03	1.214733e+02	2
7	Rastrigin	CUDA GPU	4.566724e+03	1.214733e+02	
8	Rastrigin	OpenMP	4.566724e+03	1.214733e+02	2
9	Rosenbrock	CPU	2.428757e+07	5.018349e+06	
10	Rosenbrock	CUDA GPU	2.428757e+07	5.018349e+06	
11	Rosenbrock	OpenMP	2.428757e+07	5.018349e+06	
12	Schwefel	CPU	1.628619e+05	3.762138e+03	2
13	Schwefel	CUDA GPU	1.628619e+05	3.762138e+03	
14	Schwefel	OpenMP	1.628619e+05	3.762138e+03	2
15	Sphere	CPU	4.572712e+04	3.817332e+03	
16	Sphere	CUDA GPU	4.572712e+04	3.817332e+03	
17	Sphere	OpenMP	4.572712e+04	3.817332e+03	

	function	CPU mean seconds	OpenMP mean seconds	CUDA mean seconds	Speedup CPU/OpenMP	Speedup CPU/CUDA	CP
0	Ackley	25.583220	20.846066	1.522846	1.227244	16.799610	1.6003
1	Griewank	32.778169	23.867249	2.095446	1.373353	15.642577	4.2406
2	Rastrigin	25.663944	20.620848	1.546399	1.244563	16.595935	4.5667
3	Rosenbrock	8.110166	6.964398	1.198633	1.164518	6.766182	2.4287
4	Schwefel	27.692223	21.632671	1.694492	1.280111	16.342488	1.6286
5	Sphere	7.985455	6.683950	1.136226	1.194721	7.028049	4.5727

Next steps: Generate code with `summary` New interactive sheet

6. Comparison graphs

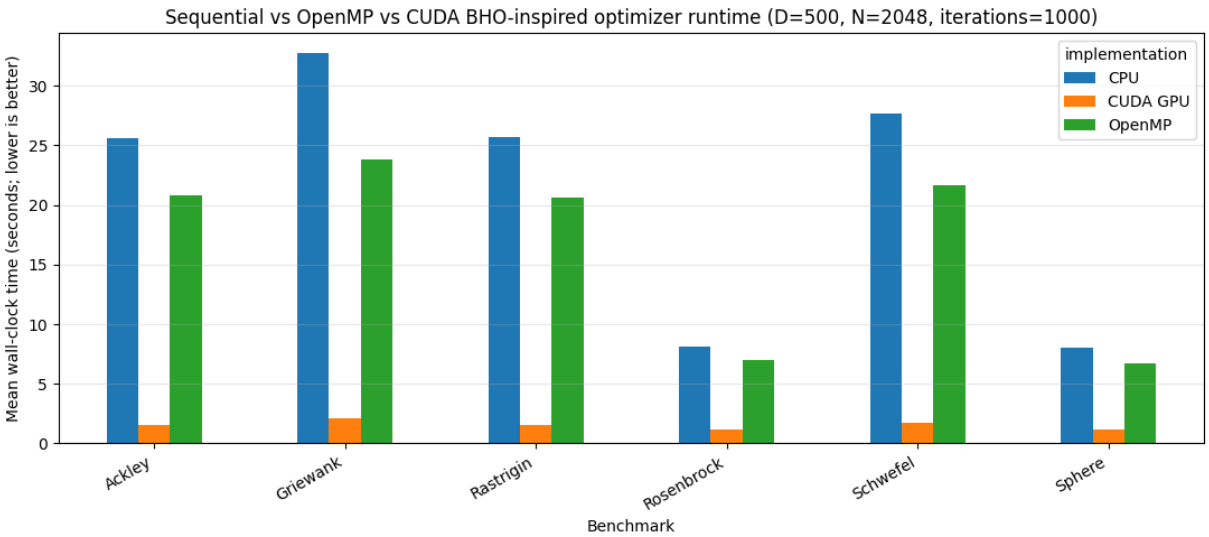
The score plots are separated by benchmark because their objective-value scales differ substantially.

```
ax = times.plot(kind="bar", figsize=(11, 5))
ax.set_title(f"Sequential vs OpenMP vs CUDA BHO-inspired optimizer r
ax.set_ylabel("Mean wall-clock time (seconds: lower is better)")
```

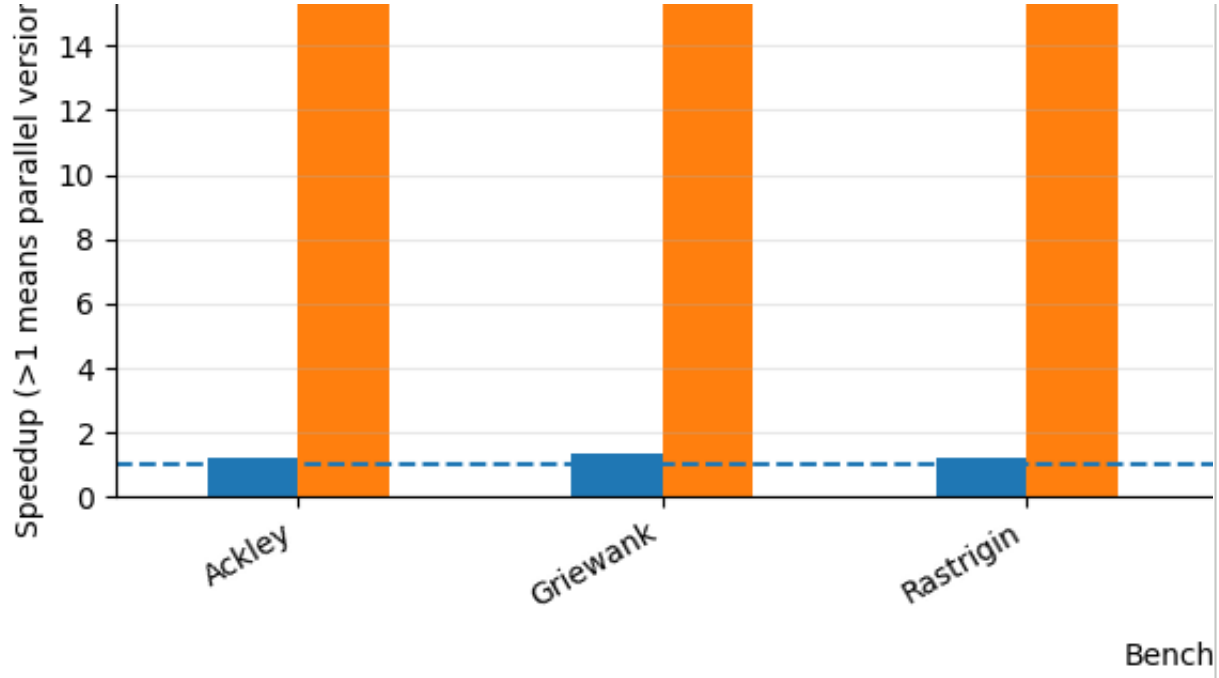
```
ax.set_xlabel("Benchmark")
ax.grid(axis="y", alpha=0.3)
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()

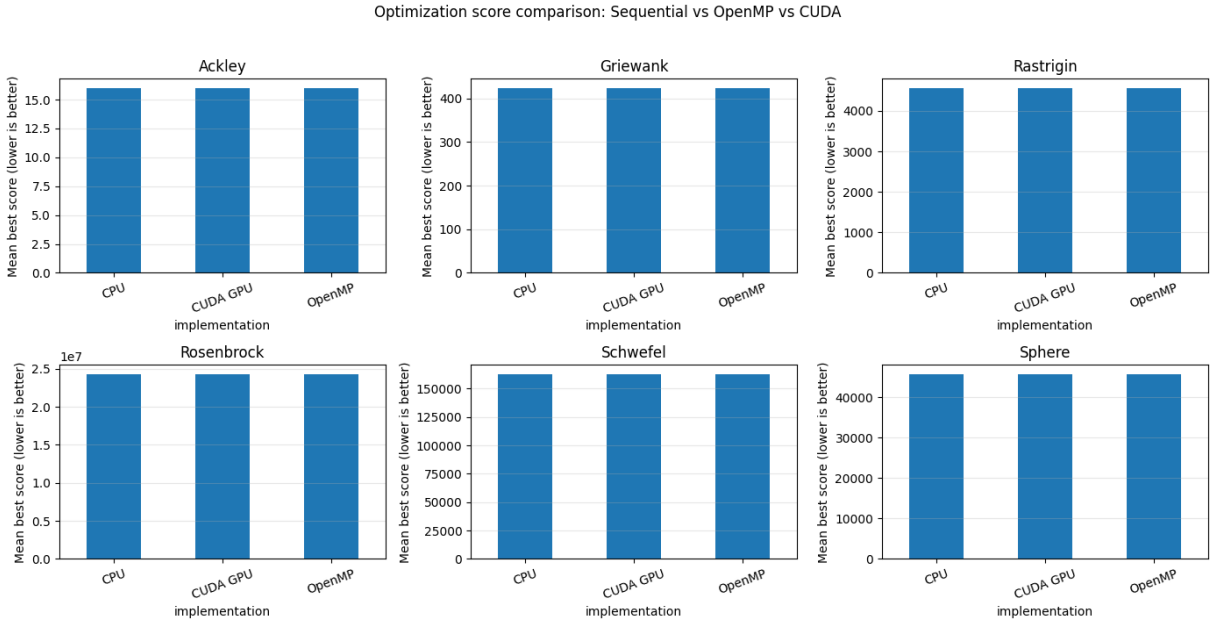
ax = comparison[["Speedup CPU/OpenMP", "Speedup CPU/CUDA"]].plot(kin
ax.axhline(1.0, linestyle="--")
ax.set_title("Parallel speedup (sequential CPU time / parallel time)
ax.set_ylabel("Speedup (>1 means parallel version faster)")
ax.set_xlabel("Benchmark")
ax.grid(axis="y", alpha=0.3)
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()

functions = list(scores.index)
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, fn in zip(axes.flat, functions):
    scores.loc[fn].plot(kind="bar", ax=ax)
    ax.set_title(fn)
    ax.set_ylabel("Mean best score (lower is better)")
    ax.tick_params(axis="x", rotation=20)
    ax.grid(axis="y", alpha=0.3)
for ax in axes.flat[len(functions):]:
    ax.set_visible(False)
fig.suptitle("Optimization score comparison: Sequential vs OpenMP vs
plt.tight_layout()
plt.show()
```






Speedup (>1 means parallel versior
14
12
10
8
6
4
2
0
Ackley
Griewank
Rastrigin
Bench


Optimization score comparison: Sequential vs OpenMP vs CUDA
Ackley
Griewank
Rastrigin
Rosenbrock
Schwefel
Sphere
Mean best score (lower is better)
implementation
CPU
CUDA GPU
OpenMP
1e7

7. Scaling experiments and reporting

- First verify the run at `D=30`, `N=256`.
- Then try `D=100`, `N=512` and `D=300`, `N=1024` if runtime allows.
- Keep `ITERATIONS` and `REPEATS` identical between Sequential, OpenMP, and CUDA.
- Set `OMP_THREADS = 4` in the benchmark cell to control the number of OpenMP CPU threads.
- Report CPU/GPU model, thread count, dimension, population, iterations, repeats, mean runtime, speedup, and mean/std best score.
- The best score is stochastic and may not match the known optimum. Runtime alone does not establish optimization quality.
- The CUDA implementation is GPU-resident, while the OpenMP version uses shared-memory CPU parallelism. Compare end-to-end wall-clock time.
- Note: these implementations are a BHO-inspired comparison variant, not a line-by-line reproduction of every mechanism in the paper (K-means, QPRS, and Explorpolis).